

Relatório da Unidade 2 — Sistema de Gerenciamento de Biblioteca em C

1. Introdução

Nesta segunda unidade, o projeto foi expandido para incluir novas ferramentas e conceitos da programação estruturada, com foco em **manipulação de strings, ponteiros, matrizes e alocação dinâmica de memória**.

A proposta foi elevar o nível técnico do código, tornando-o mais modular, eficiente e próximo de aplicações reais, sem perder a simplicidade e a clareza necessárias para o aprendizado.

Os **objetivos da Unidade 2** foram:

- Implementar **ponteiros e passagem por referência**, garantindo operações seguras e diretas sobre os dados.
- Aplicar **alocação dinâmica de memória** para possibilitar expansão do sistema sem limitação fixa.
- Aprimorar o tratamento de **strings e matrizes**, reforçando a robustez das entradas e saídas de texto.

Entre as novas **funcionalidades implementadas**, destacam-se: o cadastro dinâmico de usuários e livros, a busca textual otimizada, o controle de empréstimo e devolução com atualização automática de status e o uso de ponteiros para manipulação direta de registros na memória.

2. Metodologia

2.1 Ferramentas Utilizadas

O ambiente de desenvolvimento foi mantido o mesmo da Unidade 1, garantindo consistência e compatibilidade:

- **Linguagem:** C
- **Compilador:** GCC (via MinGW/MSYS2) – versão 15.2.0
- **IDE:** Visual Studio Code
- **Sistema de versionamento:** GitHub

2.2 Abordagem de Desenvolvimento

O desenvolvimento seguiu uma **abordagem incremental**, na qual as novas funcionalidades foram incorporadas de forma progressiva, testadas individualmente e depois integradas ao sistema principal.

O código foi refatorado com foco na **modularidade** e **reuso**, dividindo funções em blocos independentes para cadastro, busca, empréstimo e devolução.

2.3 Organização do Código

O projeto passou de um modelo baseado em vetores paralelos para um modelo orientado por **estruturas compostas (struct)**.

Cada **struct** representa uma entidade do sistema:

```
typedef struct {  
    char titulo[100];  
    char autor[100];  
    bool emprestado;  
} Livro;
```

```
typedef struct {  
    char nome[100];  
    char email[100];  
    bool pendente;  
} Usuario;
```

Essas estruturas são manipuladas por ponteiros em funções como **cadastrarLivro()**, **emprestimo()** e **devolucao()**, permitindo **alterações diretas na memória** sem cópias desnecessárias.

Além disso, o sistema passou a utilizar **alocação dinâmica (malloc e realloc)** para expandir automaticamente a capacidade de armazenamento conforme novos cadastros são feitos.

3. Análise do Código

3.1 Strings

As strings foram amplamente utilizadas na manipulação de nomes, títulos e e-mails.

Para melhorar a robustez, o uso de `fgets()` substituiu `scanf()` em todas as entradas textuais, permitindo a leitura de espaços e prevenindo falhas de buffer. Funções como `strcmp()` e `strcpy()` foram empregadas em validações e cópias de dados, respectivamente.

Exemplo:

```
if (strcmp(usuario.email, emailDigitado) == 0) {  
    printf("Usuário encontrado.\n");  
}
```

Essa abordagem reforça a confiabilidade das buscas e reduz o risco de inconsistências nas entradas do operador.

3.2 Estruturas de Repetição Aninhadas

Os laços aninhados foram aplicados principalmente em **buscas cruzadas** — como identificar se um determinado usuário possui um livro emprestado.

Um exemplo típico ocorre no processo de devolução:

```
for (int i = 0; i < totalUsuarios; i++) {  
    for (int j = 0; j < totalLivros; j++) {  
        if (strcmp(usuario[i].email, emailDigitado) == 0 && livros[j].emprestado) {  
            // lógica de devolução  
        }  
    }  
}
```

Essa estrutura garante percorrimento eficiente entre as duas bases (usuários e livros).

3.3 Matrizes

As matrizes foram implementadas principalmente para armazenar **relatórios tabulares** e **históricos de empréstimos**, simulando uma visão estruturada dos dados.

Um exemplo foi a matriz de registros:

```
int historico[totalUsuarios][totalLivros];
```

Cada linha representa um usuário e cada coluna, um livro, armazenando **1** para empréstimo ativo e **0** para devolvido.

O percorrimento é feito com dois loops aninhados, garantindo controle total sobre as combinações possíveis.

Essa implementação demonstrou o potencial das matrizes para representar relações entre entidades do sistema.

3.4 Ponteiros

O uso de **ponteiros** foi um marco na evolução da Unidade 2. Eles foram empregados para permitir que funções alterassem diretamente os dados das estruturas.

Exemplo:

```
void emprestarLivro(Livro *livro, Usuario *usuario) {  
    livro->emprestado = true;  
    usuario->pendente = true;  
}
```

Essa estratégia eliminou a necessidade de retorno de múltiplos valores e simplificou a passagem de estruturas grandes entre funções.

Além disso, o uso de ponteiros melhorou a **eficiência**, pois evita cópias desnecessárias de blocos de memória.

3.5 Alocação Dinâmica

A alocação dinâmica foi introduzida para permitir expansão do acervo e da base de usuários sem limites fixos:

```
livros = realloc(livros, (totalLivros + 1) * sizeof(Livro));
```

O tratamento de falhas de alocação foi implementado com verificações:

```
if (livros == NULL) {  
    printf("Erro: memória insuficiente.\n");  
    exit(1);  
}
```

A memória é liberada com **free()** ao final da execução, garantindo **ausência de vazamentos (memory leaks)**.

Essa abordagem torna o sistema escalável, eficiente e compatível com práticas reais de desenvolvimento em C.

4. Dificuldades e Soluções

As principais dificuldades enfrentadas nesta unidade estiveram relacionadas ao **gerenciamento de memória** e ao **uso de ponteiros**.

Erros comuns, como acesso indevido a posições de memória e falhas em `realloc`, exigiram atenção especial ao controle de índices e validações de ponteiro nulo.

A solução foi a implementação de verificações antes de cada acesso e a padronização da liberação de memória.

Outro desafio foi compreender o comportamento das **strings** com `fgets()` e `strcmp()`, que exigem cuidado com o caractere de quebra de linha `\n`.

O aprendizado prático mostrou que **testes incrementais** são essenciais para depurar erros em programas com ponteiros e estruturas complexas.

5. Conclusão

A Unidade 2 representou um salto significativo na maturidade técnica do projeto. Os conceitos de **ponteiros, alocação dinâmica e estruturas compostas** trouxeram robustez e flexibilidade ao sistema, aproximando-o de aplicações reais.

Comparado à Unidade 1, o código passou de um modelo linear e fixo para uma arquitetura **modular, expansível e orientada a dados**.

O principal aprendizado foi entender como o gerenciamento manual de memória impacta diretamente a estabilidade do programa.

Para a próxima unidade (U3), os esforços serão voltados à **implementação de persistência de dados em arquivos, tratamento de exceções, e melhorias na interface com o usuário**, consolidando o sistema como uma aplicação completa e funcional.

Respostas às Perguntas Orientadoras

- **Conceitos da U2 aplicados:** manipulação de strings, estruturas compostas, ponteiros, alocação dinâmica e matrizes.
- **Organização do código:** modular e estruturada em funções independentes, o que facilita manutenção e legibilidade.
- **Principais desafios técnicos:** manipulação de memória dinâmica e controle de ponteiros.
- **Implementação de matrizes:** usada para relacionar usuários e livros em relatórios.

- **Gerenciamento de memória:** com `malloc`, `realloc` e `free`, garantindo checagem de falhas.
- **Prevenção de vazamentos:** liberação ordenada de todos os blocos de memória no final do programa.
- **Vantagens da alocação dinâmica:** expansão ilimitada do acervo e redução de desperdício de memória.
- **Uso de ponteiros para eficiência:** permite manipular diretamente os dados das estruturas, economizando tempo e recursos.